

Analysis of Prompt Engineering Techniques and Their Algorithmic Effect on Large Language Model Output

1st Sarwagya Acharya

Department of Electronics and Computer Engineering
National College of Engineering, Tribhuvan University
Kathmandu, Nepal
formalsarwagya@gmail.com

2nd Nitesh Pant

Department of Electronics and Computer Engineering
National College of Engineering, Tribhuvan University
Kathmandu, Nepal
EMAIL_NITESH@ncet.edu.np

3rd Brishav Joshi

Department of Electronics and Computer Engineering
National College of Engineering, Tribhuvan University
Kathmandu, Nepal
EMAIL_BRISHAV@ncet.edu.np

Abstract—Prompt engineering has rapidly evolved from ad-hoc text tweaks into a family of techniques that systematically reshape how large language models (LLMs) generate output. Yet the literature largely treats these techniques as linguistic patterns, leaving their algorithmic consequences under-quantified. This paper analyses prompt engineering techniques through the lens of the conditional distribution that an LLM defines over output tokens, and characterises how each technique shifts that distribution via prompting, decoding, and reasoning-trace construction. We taxonomise the major technique families, formalise their effect on sampling, attention, and output diversity, and report a controlled empirical comparison across representative tasks. The results show that the algorithmic effect of a prompt often dominates its lexical content, and we recommend evaluation protocols that make this effect measurable.

Index Terms—prompt engineering, large language models, chain-of-thought, decoding strategies, evaluation, algorithmic analysis.

I. INTRODUCTION

Large language models (LLMs) are now routinely deployed for question answering, code synthesis, summarisation, and tool use, and the practical quality of an LLM application is shaped as much by the prompt that surrounds a model call as by the model weights themselves [1], [2]. A growing catalogue of *prompt engineering* techniques—zero- and few-shot exemplars, chain-of-thought (CoT) [3], self-consistency [4], tree-of-thoughts (ToT) [5], ReAct [6], and retrieval-augmented prompts [7]—reports substantial gains on established benchmarks, often with no change to the underlying model.

The dominant framing in this literature is empirical and qualitative: a technique is described by its prompting template and illustrated with selected outputs. While useful, this framing obscures a more fundamental question, namely *what does the technique change inside the model?* In this paper we argue that the principal techniques are best understood as algorithmic

operations on the conditional distribution $p_\theta(y \mid P, x)$ that an LLM defines over output tokens. Under this view, a prompt is not merely a textual wrapper; it is a control input that perturbs sampling, restructures attention, alters logit geometry, and conditions intermediate computational traces. The size of the resulting behavioural change often rivals, and sometimes exceeds, the difference between adjacent model scales.

The contributions of this paper are as follows.

- A taxonomy that organises prompt engineering techniques by the algorithmic object they modify: the prompt template, the decoding policy, or the reasoning trace.
- A formal framework, presented in Part 2, that expresses each technique family as a transformation on $p_\theta(y \mid P, x)$ and on the underlying $\arg \max$ or sampling operator.
- A controlled empirical study that isolates the algorithmic effect of prompting from the lexical effect by holding the prompt template fixed and varying decoding parameters (T, k, p), and vice versa.
- An evaluation protocol that we recommend for future prompt engineering research, designed to make algorithmic and lexical contributions separable.

The remainder of this paper is organised as follows. Section II reviews prior surveys and the foundational techniques that motivate this work. Section III (Part 2) introduces the taxonomy and notation. Section IV (Part 2) formalises the algorithmic effect of each technique. Section V (Part 2) describes the experimental design. Section VI (Part 3) reports the results, and Sections VII–IX (Part 3) discuss them, list limitations, and conclude.

II. BACKGROUND AND RELATED WORK

The prompt engineering literature spans prompting patterns, decoding-time methods, and reasoning-trace construction. We

organise the review along the same three axes used in our taxonomy (Part 2, Section III).

A. Prompting patterns as control inputs

Brown et al. [1] demonstrated that scaling model size produces *in-context learning*, in which a small set of exemplars in the prompt elicits task behaviour without weight updates. The result established prompting as a first-class interface to LLMs and motivated subsequent work on exemplar selection, ordering, and formatting. Min et al. [2] further showed that the ground-truth labels in exemplars contribute less than the input–output format, evidence that prompts act partly as structural scaffolds for the model’s own computations.

B. Reasoning-trace construction

Wei et al. [3] introduced chain-of-thought prompting, appending intermediate reasoning steps to exemplars and observing large gains on arithmetic, commonsense, and symbolic tasks. The technique reframes generation as a two-stage process: sample a reasoning chain r , then sample the answer y conditioned on r . Wang et al. [4] extended this with self-consistency, sampling multiple chains and selecting the most frequent final answer; the gain over greedy CoT demonstrates that the underlying distribution is multimodal and that marginalising over reasoning paths reallocates probability mass towards correct answers. Yao et al. [5] generalised chains to trees (tree-of-thoughts), and Wei et al. [6] interleaved reasoning with tool calls in ReAct. Across these works, the common algorithmic move is to expose intermediate computation to the sampling operator.

C. Decoding-time and retrieval augmentation

Beyond reasoning traces, decoding-time methods such as temperature, top- k , and nucleus (top- p) sampling [8] directly parametrise the output distribution. Retrieval-augmented generation (RAG) couples the LLM to an external retriever, conditioning generation on documents fetched at inference time. While RAG is often presented as a knowledge-injection technique, it is, in our framing, a prompting technique: it changes the conditioning context P of $p_\theta(y \mid P, x)$ and thereby changes the posterior over outputs.

D. Positioning of this paper

Prior surveys catalogue prompting techniques but do not formalise their algorithmic effect. Empirical comparisons exist for individual techniques, but typically vary the prompt while holding decoding fixed, confounding the algorithmic and lexical contributions. To our knowledge, no prior work isolates the algorithmic effect of prompting through controlled ablations on both the prompt template and the decoding policy. This paper fills that gap by treating prompting and decoding as two parameters of the same sampling operator and measuring each contribution independently.

E. Common notation and primer

Throughout the paper we write P for a prompt template, x for the user input, and y for the model output. An LLM with parameters θ defines a conditional distribution $p_\theta(y \mid P, x)$. Decoding is parameterised by a temperature T , a top- k cutoff, and a nucleus threshold p . We treat CoT, self-consistency, and ToT as sampling procedures over reasoning traces r followed by answers y , and we treat retrieval augmentation as a stochastic rewrite of P . These objects are formalised in Part 2, Section IV.

III. TAXONOMY OF PROMPT ENGINEERING TECHNIQUES

To be authored in Part 2.

IV. ALGORITHMIC FRAMEWORK

To be authored in Part 2.

V. EXPERIMENTAL SETUP

To be authored in Part 2 (methodology) and Part 3 (final parameters).

VI. RESULTS AND ANALYSIS

To be authored in Part 3.

VII. DISCUSSION

To be authored in Part 3.

VIII. LIMITATIONS AND THREATS TO VALIDITY

To be authored in Part 3.

IX. CONCLUSION

To be authored in Part 3.

ACKNOWLEDGMENT

To be added before submission.

REFERENCES

- [1] T. Brown et al., “Language models are few-shot learners,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
- [2] S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer, “Rethinking the role of demonstrations: What makes in-context learning work?,” in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2022, pp. 10748–10760.
- [3] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 24824–24837.
- [4] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [5] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 11809–11822.
- [6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [7] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [8] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The curious case of neural text degeneration,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.